

ECOM EXPERTS

Samestock

Manual de usuario, técnico y de desarrollo

samestock 1.11.0 · 07/09/2026

Contenido

1. ¿Para qué sirve?

2. Después de actualizar: haz esto una vez

3. Dónde está la pantalla

4. Asociar productos

La asociación es siempre de ida y vuelta (y es lo correcto)

5. Asociar combinaciones

Cambiar una asociación

Quitar una asociación

6. Cadenas de varios productos

7. Configuración del módulo

Pestaña Sincronización

Pestaña Depuración

8. Mantenimiento

9. Preguntas frecuentes

10. Si algo no funciona

1. Qué hace

2. Estructura de ficheros

3. Modelo de datos

PREFIX_samestock_products

PREFIX_samestock_combis

4. Hooks

Auto-registro de hooks (autoRegisterHooks())

Página de producto V1 vs V2 (crítico)

Detección de V1 / V2: en ejecución, no por configuración

Pestaña propia en la V2

5. Transporte del formulario (por qué es así)

Sin dependencias de JavaScript

6. Guardado: limpiar y reasociar, nunca actualizar

Validación de coherencia (defensa en el servidor)

7. Propagación de stock

Multitienda

8. Configuración

9. Migraciones

10. Sistema de depuración

11. Notas de compatibilidad verificadas contra el core

Propagación de stock en la página V2

API por versión

Limitaciones conocidas

Síntomas reportados

Causa 1 — El JS concatenaba el id en el name en vez de reemplazarlo

Causa 2 — El autocompletar ofrecía las combinaciones de todos los productos

Causa 3 — El DELETE de guardado usaba la columna equivocada

Causa 4 — Comprobación anti-duplicado tautológica

Causa 5 — Colisión de claves en las filas nuevas de productos

Causa 6 — Recorrido de propagación escrito a mano y de sólo 2 niveles

Fallos adicionales corregidos por el camino

Segunda revisión: dos blockers más de la página V2

Blocker A — ¿llegaba siquiera el hook de stock en la V2?

Blocker B — jQuery UI no está garantizado en la V2

Riesgo cerrado — actionProductUpdate de otro producto

Verificado también para PS 1.7.8

Detección de la página de producto, endurecida

Tercera revisión: "no sale nada en la ficha de producto"

Causa — el hook no estaba registrado y en V2 eso significa no existir

Y el sitio donde aparecía tampoco era el bueno

Cuarta revisión: seguía sin salir nada

Causa raíz — la detección de V1/V2 se deducía de la configuración

Un camino que se descartó (y por qué)

Panel de diagnóstico

Compatibilidad PS 8 / PS 9: el blocker que había que resolver

La trampa del envío parcial en V2

Verificación realizada

Si algo falla en producción

Compartir el stock entre varios productos y entre sus combinaciones

Versión 1.6.4 · Compatible con PrestaShop 1.7, 8.x y 9.x

1. ¿Para qué sirve?

Para vender el mismo artículo desde varias fichas de producto sin duplicar el inventario. Cuando se vende una unidad en cualquiera de las fichas asociadas, el stock baja en todas.

Funciona en dos niveles:

- **Productos completos:** dos o más fichas sin combinaciones comparten la misma cantidad.
- **Combinaciones concretas:** la talla M de la ficha A comparte stock con la talla M de la ficha B. Se decide pareja por pareja.

2. Después de actualizar: haz esto una vez

1. Ve a **Módulos** → **Gestor de módulos**, busca *samestock* y pulsa **Configurar**.
2. En el panel **Mantenimiento**, pulsa **Limpiar asociaciones incoherentes**.

Esto elimina las asociaciones que quedaron mal guardadas o mezcladas con las versiones anteriores. Si no lo haces, pueden seguir apareciendo combinaciones cruzadas de antes.

Después revisa las asociaciones de tus productos: las que hubieran quedado corruptas aparecerán vacías y bastará con volver a elegir las.

3. Dónde está la pantalla

Depende de la versión de tu tienda y de si usas la página de producto nueva o la clásica:

Situación	Dónde aparece
PrestaShop 1.7 / 8.0, o 8.1-8.2 con la página de producto clásica	pestaña Ajustes (productos asociados) y pestaña Cantidades (combinaciones)
PrestaShop 8.1 / 8.2 con la página de producto nueva , y PrestaShop 9	pestaña propia Stock compartido

En PrestaShop 8.1+ y 9 el módulo añade su propia pestaña **Stock compartido** a la ficha de producto, junto a Descripción, Precio, Existencias, etc. Los dos bloques (productos y combinaciones) están ahí dentro.

Si por alguna razón el JavaScript del back-office no se cargara, el contenido seguiría accesible en la pestaña **Módulos** del producto. Funciona igual; sólo cambia el sitio.

4. Asociar productos

1. Abre el producto y ve al bloque **Productos con stock compartido**.
2. Pulsa **Agregar producto**.
3. Empieza a escribir el nombre y **elige uno de la lista** que aparece.
4. **Guarda el producto**.

Notas importantes:

- Hay que **elegir de la lista**. Si escribes un nombre a mano y no coincide con ningún producto, la fila se marca y se ignora al guardar.
- El icono de papelera quita la fila. Se aplica al **guardar**.
- Los iconos de flecha y de ojo abren la ficha del producto asociado en el back-office y en la tienda.

La asociación es siempre de ida y vuelta (y es lo correcto)

Si desde el producto A asocias el producto B, al abrir **B verás A** en su lista. No es un error: es cómo tiene que ser.

«Mismo stock» significa que A y B comparten **una sola** cantidad real. Si la relación fuera de un solo sentido, al vender una unidad por B el stock de A no bajaría y acabarías vendiendo unidades que no tienes. Por eso el módulo crea siempre las dos direcciones, y por eso una unidad vendida en cualquiera de los dos descuenta en ambos.

Para **romper** la relación da igual desde cuál de los dos lo hagas: quitar la fila en A o en B elimina la asociación completa.

5. Asociar combinaciones

Primero hay que guardar la asociación de productos. El módulo necesita saber qué productos están asociados para poder ofrecerte sus combinaciones.

1. Guarda el producto con los productos asociados que quieras.
2. Vuelve a abrirlo y ve al bloque **Combinaciones con stock compartido**.
3. Verás un apartado por cada producto asociado, y dentro una línea por cada combinación de **este** producto.
4. En cada línea hay un **desplegable** con las combinaciones del producto asociado. Elige la que debe compartir el stock. El desplegable sólo contiene combinaciones de ese producto, así que no puedes mezclarlas por error.
5. **Guarda el producto**.

Cambiar una asociación

Elige simplemente otra opción del desplegable. Al guardar, el módulo **elimina** la asociación anterior de esa línea y crea la nueva. No se acumulan.

Si eliges una combinación que ya estaba usada por otra línea, esa otra línea queda libre: una combinación de destino sólo puede estar emparejada con un origen. Es intencionado, y evita que dos combinaciones distintas peleen por el mismo stock.

Quitar una asociación

Deja el desplegable en «— **Sin asociación** —» y guarda.

6. Cadenas de varios productos

Las asociaciones se propagan en cadena. Si asocias A con B, B con C y C con D, un cambio de stock en A llega a B, C y D. No hace falta asociar A con todos: basta con encadenarlos.

Igual con las combinaciones: si la talla M de A está ligada a la de B, y la de B a la de C, un cambio en cualquiera de las tres actualiza las otras dos.

7. Configuración del módulo

Módulos → Gestor de módulos → samestock → Configurar

Pestaña Sincronización

- **Recalcular el stock total del producto padre** (*recomendado: Sí*) Tras propagar una combinación, recalcula el stock total de cada producto afectado como la suma de sus combinaciones. Sin esto, el total que se muestra puede quedar desfasado respecto a las combinaciones.
- **Propagar el stock de los productos sin combinaciones** (*recomendado: Sí*) Comparte el stock entre productos asociados que no tienen combinaciones.

Pestaña Depuración

- **Modo depuración** (*recomendado: No, salvo que estés investigando algo*) Escribe un fichero de registro diario en `modules/samestock/logs/` con cada guardado y cada movimiento de stock. Útil para diagnosticar, pero deja de usarlo cuando termines: los ficheros crecen.

Si tienes **multitienda**: estas tres opciones son **globales**, se aplican a todas las tiendas. La propia pantalla te lo avisa.

8. Mantenimiento

En la misma pantalla de configuración, debajo del formulario:

- **Limpiar asociaciones incoherentes** — elimina asociaciones a productos o combinaciones que ya no existen, filas duplicadas, y combinaciones ligadas a un producto que ya no está asociado. Puedes ejecutarlo cuando quieras: no borra asociaciones válidas.
- **Ficheros de log de depuración** — lista los registros y permite borrarlos.

9. Preguntas frecuentes

No veo el bloque en la ficha del producto. Comprueba en qué pestaña debe estar según tu versión (apartado 3). Si sigues sin verlo, entra en la configuración del módulo: al abrirla se registran automáticamente los hooks que

falten. Luego vacía la caché de PrestaShop (*Parámetros avanzados* → *Rendimiento* → *Vaciar caché*).

He asociado un producto pero no puedo elegir sus combinaciones. Guarda primero el producto. La lista de combinaciones disponibles se calcula al cargar la página. Desde la 1.6.4 el bloque **Combinaciones con stock compartido** aparece siempre que el producto tenga combinaciones y, si no hay nada que emparejar, te dice por qué (no hay producto asociado todavía, o el producto asociado no tiene combinaciones).

¿Por qué al abrir el otro producto aparece el primero? Es correcto y necesario: la asociación es de ida y vuelta. Ver el apartado 4.

Elijo una combinación y al guardar aparece otra. Era el fallo de las versiones anteriores, ya corregido: las combinaciones se eligen ahora con un desplegable, no escribiendo. Ejecuta **Limpiar asociaciones incoherentes** una vez y vuelve a elegirla.

En la ficha de producto no aparece nada del módulo (PrestaShop 8.1+ o 9). Era el caso de las versiones anteriores: PrestaShop sólo deja hueco a los módulos en la ficha de producto si están registrados en el hook correspondiente, y al actualizar desde una versión antigua ese registro no existía. Desde la 1.6.4 el módulo se registra solo la primera vez que se carga el back-office. Si aún no lo ves:

1. Vacía la caché (*Parámetros avanzados* → *Rendimiento* → *Vaciar caché*).
2. Recarga la ficha del producto.
3. Si sigue sin salir, entra una vez en la configuración del módulo y vuelve a probar.

El desplegable de combinaciones sale vacío. El producto asociado no tiene combinaciones creadas. Créalas primero en su ficha.

El stock total del producto no cuadra con la suma de sus combinaciones. Activa **Recalcular el stock total del producto padre** y modifica cualquier combinación para forzar el recálculo.

Un producto con combinaciones no recibe el stock de un producto sin combinaciones. Es correcto y deliberado: el total de un producto con combinaciones lo determina la suma de sus combinaciones, no un valor plano. Asocia las combinaciones entre sí.

¿Se puede compartir stock entre tiendas distintas en multitienda? El módulo propaga dentro de las tiendas donde el producto tiene stock. Si necesitas un comportamiento distinto entre tiendas, coméntalo.

10. Si algo no funciona

1. Configuración del módulo → **Depuración** → activar **Modo depuración** → Guardar.
2. Reproduce el problema: asocia, elige las combinaciones y guarda.
3. Envía el fichero `modules/samestock/logs/samestock-AAAA-MM-DD.log`.

Ese registro dice exactamente qué recibió el módulo del formulario y qué guardó, lo que permite localizar el problema sin adivinar.

samestock — Documentación técnica

Versión: 1.6.4 **Autor:** Ecom Experts ecomyseo@gmail.com **Licencia:** AFL-3.0 **Compatibilidad:** PrestaShop 1.7 / 8.0 / 8.1 / 8.2 / 9.x — PHP 7.4 y 8.x

1. Qué hace

Comparte el stock entre varios productos y entre combinaciones concretas de esos productos. Cuando cambia la cantidad de un producto o de una combinación, el módulo propaga esa cantidad a todos los nodos ligados.

2. Estructura de ficheros

```
samestock/  
  samestock.php           clase principal (namespace global, sin Composer)  
  classes/  
    SameStockRepository.php  toda la lógica de datos y de grafo  
    SameStockLogger.php      sistema de depuración a fichero  
  controllers/              (no se usa: el módulo no tiene controladores propios)  
  views/  
    js/samestock.js         UI de asociación producto ↔ producto (JS nativo)  
    css/samestock.css        estilos propios, prefijo samestock-  
    templates/admin/listaproductos.tpl  
    templates/admin/listadocombis.tpl  combinaciones con <select> nativo, sin JS  
    templates/admin/maintenance.tpl    panel de mantenimiento y logs  
  translations/es-ES.xlf     sistema de traducción NUEVO  
  sql/install.php, sql/uninstall.php  
  logs/                      ficheros .log (protegidos con .htaccess)  
  index.php                  en cada directorio, recursivo
```

No existen `src/`, `config/`, `composer.json`, `vendor/` ni `upgrade/`. Las migraciones viven en `addnewfeatures()`.

3. Modelo de datos

`PREFIX_samestock_products`

Columna	Tipo	Descripción
<code>id_samestock</code>	int AI	PK
<code>id_product</code>	int	producto origen
<code>id_pro</code>	int	producto asociado
<code>id_asoc</code>	int	heredada, sin uso

Índices: `UNIQUE (id_product, id_pro) — samestock_products_unique ; KEY (id_pro) .`

La relación es **simétrica**: existen siempre las dos filas `(A,B)` y `(B,A)` .

`PREFIX_samestock_combis`

Columna	Tipo	Descripción
<code>id</code>	int AI	PK
<code>id_product</code>	int	producto origen
<code>id_product_attribute</code>	int	combinación origen
<code>id_pro</code>	int	producto asociado
<code>id_pro_attribute</code>	int	combinación asociada
<code>id_asoc</code>	int	heredada, sin uso

Índices:

- `UNIQUE (id_product, id_product_attribute, id_pro) — samestock_combis_source` : una combinación origen sólo puede apuntar a **una** combinación de cada producto asociado.
- `UNIQUE (id_product, id_pro, id_pro_attribute) — samestock_combis_target` : una combinación destino sólo puede estar espejada por **un** origen del mismo producto.
- `KEY (id_pro, id_pro_attribute) — samestock_combis_reverse` , para el recorrido inverso.

Los dos índices `UNIQUE` son la garantía a nivel de base de datos de que las combinaciones no pueden mezclarse.

`samestock_combis_combis` y `samestock_combisstock` se siguen creando por compatibilidad con instalaciones antiguas, pero el módulo no las usa.

4. Hooks

Hook	Método	Para qué
<code>displayBackOfficeHeader</code>	<code>hookDisplayBackOfficeHeader</code>	carga JS y <code>Media::addJsDef</code> (<code>pros</code> , <code>combisposibles</code>)
<code>displayAdminProductsMainStepLeftColumnBottom</code>	idem	bloque de productos — sólo página de producto V1
<code>displayAdminProductsQuantitiesStepBottom</code>	idem	bloque de combinaciones — sólo V1
<code>displayAdminProductsExtra</code>	<code>hookDisplayAdminProductsExtra</code>	contenido de los dos bloques — sólo página de producto V2
<code>actionProductFormBuilderModifier</code>	<code>hookActionProductFormBuilderModifier</code>	crea la pestaña propia — sólo V2
<code>actionProductUpdate</code>	idem	guardado de las asociaciones
<code>actionUpdateQuantity</code>	idem	propagación de stock
<code>actionProductDelete</code>	idem	limpia asociaciones al borrar el producto
<code>actionProductAttributeDelete</code>	idem	borrado de todas las combinaciones
<code>deleteProductAttribute</code>	<code>hookDeleteProductAttribute</code>	borrado de una combinación

La lista está centralizada en `Samestock::getHookList()`, que usan `install()`, `addnewfeatures()` y el auto-registro. Todos los hooks registrados tienen su método.

Auto-registro de hooks (`autoRegisterHooks()`)

Se ejecuta desde el constructor cuando el módulo se instancia en el back-office.

Es necesario, no una comodidad: en la página V2 el módulo **sólo aparece si está registrado en `displayAdminProductsExtra`**, porque `ProductTypeListener` elimina el campo `extra_modules` del formulario cuando ningún módulo implementa ese hook. Al actualizar desde una versión antigua esos hooks no existen todavía, así que sin auto-registro no se ve **nada** en la ficha de producto hasta que alguien abre la configuración del módulo.

Coste: se guarda la versión ya registrada en `SAMESTOCK_HOOKS_VERSION`; en cuanto coincide, la comprobación es una lectura de `Configuration` (ya en caché). Todo dentro de `try/catch (Throwable)`: nunca puede romper la instanciación del módulo.

Página de producto V1 vs V2 (crítico)

`displayAdminProductsMainStepLeftColumnBottom` y `displayAdminProductsQuantitiesStepBottom` son hooks de la página de producto **clásica (V1)**. Verificado contra el código fuente:

Versión	V1 disponible	Hooks V1 renderizados	<code>displayAdminProductsExtra</code>	<code>actionProductFormBuilderModifier</code>
1.7.x / 8.0	sí	sí	sí	— (no hay V2)
8.1 / 8.2	sí, si el feature flag <code>product_page_v2</code> está apagado	sólo con V1	sí	sí
9.x	no, se eliminó	nunca	sí	sí

En PS 9 esos dos hooks existen en `hook.xml` (el `registerHook` funciona) pero **ninguna plantilla del core los renderiza**. Sin `displayAdminProductsExtra` el módulo no mostraría nada en PS 9.

Detección de V1 / V2: en ejecución, no por configuración

`isProductPageV2()` devuelve, por este orden:

1. `self::$v2_runtime` — la señal fiable. Se pone a `true` desde `hookActionProductFormBuilderModifier`, y que ese hook se ejecute es **prueba** de que estamos en V2, no una deducción: - el `ProductController` de V1 construye su formulario con el `createFormBuilder()` de Symfony y tiene **cero** referencias a `IdentifiableObject` (verificado en 8.2.7); - el único que despacha `action<Prefijo>FormBuilderModifier` es `Core\Form\IdentifiableObject\Builder\FormBuilder::buildForm()`; - el `ProductController` de V2 sí usa `prestashop.core.form.identifiable_object.builder.edit_product_form_builder`.

Y ocurre en tiempo de **construcción** del formulario, antes de cualquier renderizado, así que

`displayAdminProductsExtra` ya lo encuentra puesto. 2. `version_compare(_PS_VERSION_, '9.0.0', '>=')` — en PS 9 la V1 no existe. 3. `PREFIX_feature_flag.state` del flag `product_page_v2` — respaldo para 8.1/8.2. Se consulta la tabla y no un servicio de Symfony porque el nombre del servicio cambia entre versiones.

Los puntos 2 y 3 son sólo respaldo. Antes eran la única fuente, y eso era un punto único de fallo: si la deducción salía mal (tienda actualizada sin la fila del flag, o con el flag a 0 sirviendo la V2), `displayAdminProductsExtra` devolvía cadena vacía, los hooks V1 no se renderizan en V2, y **no se pintaba nada en ningún sitio**.

Los tres hooks de display se excluyen mutuamente, de modo que **nunca** se renderizan los campos dos veces. Cada uno registra en el log su decisión (`es_v2`, `v2_en_ejecucion`, `ps_version`, `pinta`).

No sirve la alternativa "pintar en `displayAdminProductsExtra` sólo si los hooks V1 no han pintado": en el `product.html.twig` de V1 el hook Extra se renderiza en la línea 33, **antes** de los paneles (líneas 68 y 91), así que cuando se ejecuta todavía no se sabe.

Pestaña propia en la V2

El módulo crea su propia pestaña **Stock compartido** en el formulario V2. Mecanismo verificado en 8.1.7, 8.2.7 y 9.1.1:

1. `EditProductFormType::getParent()` devuelve `NavigationTabType`.
2. El bloque `navigation_tab_widget` de `Admin/TwigTemplateForm/prestashop_ui_kit_base.html.twig` (línea ~1408) recorre **todos los hijos de la raíz** del formulario y convierte cada uno en pestaña: - nav: `<li id="{{ child.vars.id }}-tab-nav">` - panel: `<div id="{{ child.vars.id }}-tab">` - título: `{{ child.vars.label }}`
3. El prefijo del formulario raíz es `product` (de ahí que el core use `#product_stock-tab` en su propio `product-map.ts`).

Por tanto `hookActionProductFormBuilderModifier` añade un hijo llamado `samestock` (`FormType`, `mapped => false`, `required => false`) y el core genera de forma determinista `#product_samestock-tab`. **Ese id lo creamos nosotros**, no depende de la estructura interna del core.

El contenido lo pinta `displayAdminProductsExtra` (que cae en la pestaña *Módulos*, dentro del `<form>` porque `ExtraModulesType` es un campo de `EditProductFormType`) y `samestock.js` lo mueve a `#product_samestock-tab`. Mover el DOM no cambia el `name` de los campos y siguen dentro del mismo `<form>`, así que el envío no se ve afectado.

Degradación en cascada, sin puntos únicos de fallo:

Qué falla	Resultado
nada	pestaña Stock compartido con todo dentro
el JS no se ejecuta	contenido accesible en la pestaña Módulos + una pestaña vacía
<code>actionProductFormBuilderModifier</code> sin registrar	contenido en la pestaña Módulos
no hay contenido que mover	el JS retira la pestaña para no dejarla vacía

Se descartó a propósito trasladar el contenido a `#product_stock-tab`: `ProductTypeListener` hace `$form->remove('stock')` en varios casos, así que esa pestaña no existe siempre, y los selectores del core no son estables entre 8.1, 8.2 y 9.x. Con una pestaña propia el id es nuestro y siempre existe.

5. Transporte del formulario (por qué es así)

La página de producto V2 usa `product-partial-updater.ts`: al cargar toma una instantánea de `serializeArray()` del formulario y al guardar envía **sólo los campos cuyo valor ha cambiado** (construye un formulario "sombra" y lo envía con método `PATCH`). Eso impone tres reglas:

1. **El dato no puede ir en el `name` del campo.** Un cambio de nombre es invisible para el diff, y además fue la causa del fallo original.
2. **Una lista no puede repartirse en varios campos.** Llegaría incompleta.
3. **Los campos presentes en la instantánea y ausentes al guardar se fuerzan a ''.**

Diseño resultante:

Sección	Campo enviado	Semántica
Productos asociados	<code>samestock_products_list[ID_PRODUCTO]</code> (1 campo, CSV)	lista completa ; el JS lo recalcula al añadir/quitar filas
Combinaciones	<code>asociaciones[ORIGEN][PRODUCTO]</code> (un <code><select></code> por fila)	valor = id de la combinación destino; <code>0</code> = eliminar

Nombres **fijos**, valores **variables**. Los campos de texto visibles de la lista de productos **no tienen** `name`: sólo sirven para buscar.

La indexación por `ID_PRODUCTO` no es decorativa: `actionProductUpdate` también se lanza desde `Product::updatePosition()`, `setWsCategories()` y `updateWs()`. Si uno de esos disparos ocurriera en la misma petición para otro producto, sin la clave se aplicaría este formulario al producto equivocado.

`SameStockRepository::extractProductListForProduct()` devuelve `null` cuando la clave no corresponde, y entonces no se toca nada. Para las combinaciones la protección es implícita: `normalizeCombinationPayload()` descarta cualquier `src_pa` que no pertenezca al producto.

Si un campo no llega, esa sección no se toca:

- `samestock_products_list[ID]` ausente → no se modifica ninguna asociación de producto.
- `asociaciones` ausente → no se modifica ninguna asociación de combinación.

`SameStockRepository::normalizeCombinationPayload()` acepta además las formas antiguas (`asociaciones[ORIGEN][PRODUCTO][DESTINO]` y la corrupta con índices concatenados) por si queda JS o plantilla en caché; en ese caso toma la clave más profunda cuya hoja no esté vacía. Elegir "por hoja no vacía" y no "última clave" es lo que lo hace correcto cuando el envío parcial añade el nombre antiguo con valor vacío.

Sin dependencias de JavaScript

- **Combinaciones:** `<select>` **nativo**. Ni una línea de JS. Las opciones son sólo las combinaciones del producto asociado de esa fila, así que elegir una combinación de otro producto es imposible desde el navegador. Y al cambiar un `<select>` el navegador emite `change`, que es justo el evento que escucha el gestor de envío parcial (`keyup change dp.change` sobre el formulario) para habilitar el guardado.
- **Lista de productos:** `views/js/samestock.js` **en JavaScript nativo**, sin jQuery ni jQuery UI. La V2 no garantiza que jQuery UI esté cargado; la versión anterior usaba `$.autocomplete` y, sin él, el campo quedaba inerte y la pantalla inservible. El módulo trae su propia lista de sugerencias (`views/css/samestock.css`).
- El JS emite `change` sobre el campo oculto tras cada cambio, por el mismo motivo.

6. Guardado: limpiar y reasociar, nunca actualizar

`SameStockRepository::saveCombinationAssociations()` para cada par (combinación origen, producto asociado):

1. `DELETE` de todas las filas de esa combinación origen contra ese producto, **en los dos sentidos**.
2. Si hay destino elegido, `DELETE` de cualquier fila que ya usara esa combinación destino con otro origen del mismo producto (se libera el destino).
3. `INSERT` de las dos filas simétricas.

No hay ningún `UPDATE`. Ejecutar el mismo payload dos veces da el mismo resultado (idempotente), lo cual importa porque en PS 9 `actionProductUpdate` puede dispararse varias veces en la misma petición.

`saveProductAssociations()` hace borrado total en los dos sentidos y reconstrucción a partir de la lista recibida, y además elimina las filas de `samestock_combis` que apuntan a productos que ya no están asociados.

Validación de coherencia (defensa en el servidor)

Antes de insertar, `normalizeCombinationPayload()` comprueba contra `PREFIX_product_attribute`:

- que la combinación **origen** pertenece al producto que se está guardando;
- que la combinación **destino** pertenece al producto asociado indicado.

Lo que no cumple se descarta y se registra en el log. Con esto, aunque el JS fallara, la base de datos no puede quedar con combinaciones mezcladas entre productos.

7. Propagación de stock

`hookActionUpdateQuantity` recibe `id_product`, `id_product_attribute`, `quantity` y (en PS 9) `id_shop`.

- **Combinación** (`id_product_attribute > 0`): se calcula el **cierre transitivo** con `getLinkedCombinations()` — recorrido en anchura sobre las dos direcciones de la relación, con conjunto de visitados y tope `MAX_NODES = 300`. Se aplica la cantidad a todos los nodos alcanzados. Si `SAMESTOCK_SYNC_PARENT` está activo, se recalcula después el total (`id_product_attribute = 0`) de cada producto afectado como suma de sus combinaciones, por tienda.
- **Producto sin combinaciones** (`id_product_attribute = 0`): `getLinkedProducts()` hace el mismo recorrido y se aplica la cantidad, saltando los productos que sí tienen combinaciones (su total lo determina la suma de éstas).

La escritura se hace con SQL directo (`applyQuantity`) y no con `StockAvailable::setQuantity()`, porque `setQuantity()` volvería a disparar `actionUpdateQuantity` y provocaría recursión. El recorrido del grafo ya es completo, así que reentrar no aporta nada. Aun así existe un guard estático `self::$syncing` que corta cualquier reentrada dentro de la misma petición.

`applyQuantity()` actualiza `stock_available` y, si las columnas existen (se comprueba por `INFORMATION_SCHEMA` con caché), las columnas legacy `product_attribute.quantity` y `product.quantity`.

Multitienda

- Si el hook aporta `id_shop`, la propagación se limita a esa tienda.
- Si no, se toman las tiendas en las que existe la fila de stock del nodo origen (`getSourceShops`) y se filtra por `id_shop IN (...) OR id_shop = 0`.
- Las tres opciones de configuración son **globales** (`Configuration::getGlobalValue` / `updateGlobalValue`) y se avisa de ello en pantalla cuando `Shop::isFeatureActive()` es cierto.
- El autocompletar de productos filtra por `product_lang.id_shop` de la tienda del contexto, para no mostrar cada producto repetido una vez por tienda.

8. Configuración

Un **único** `HelperForm` con pestañas y un solo botón de guardar (`submitSamestockModule`). Todos los campos tienen su valor en `getConfigFormValues()` .

Clave	Pestaña	Defecto	Efecto
<code>SAMESTOCK_SYNC_PARENT</code>	Sincronización	1	recalcula el total del producto padre
<code>SAMESTOCK_SYNC_PRODUCTS</code>	Sincronización	1	propaga el stock de productos sin combinaciones
<code>SAMESTOCK_DEBUG</code>	Depuración	0	activa el log a fichero

`SAMESTOCK_LIVE_MODE` se conserva sólo para no romper instalaciones antiguas; no se usa ni se muestra.

Fuera del formulario, en paneles aparte:

- `maintenance.tpl` — **Limpiar asociaciones incoherentes** (`purgeOrphans()` + `migrate()`) y listado/borrado de ficheros de log.
- `diagnostics.tpl` — panel de **Diagnóstico**: versión del módulo, de PrestaShop y de PHP; **página de producto detectada (V1/V2)**; existencia de la tabla `feature_flag` y valor del flag `product_page_v2` ; multitienda; contenido de `SAMESTOCK_HOOKS_VERSION` ; si `logs/` es escribible; filas en las tablas; presencia de los índices `UNIQUE` ; y la lista de los 10 hooks con marca de registrado / no registrado.

Existe porque el síntoma "no sale nada en la ficha de producto" puede venir de varias causas distintas (hook sin registrar, V1 frente a V2, tablas sin migrar) y sin este panel la única vía era probar a ciegas. Una captura identifica la causa de inmediato.

9. Migraciones

`addnewfeatures()` se ejecuta al abrir la configuración, dentro de `try/catch` :

1. Registra los hooks que falten (incluidos `displayAdminProductsExtra` y `deleteProductAttribute`).
2. Inicializa las claves de configuración que no existan.
3. Llama a `SameStockRepository::migrate()` , que: - ejecuta `purgeOrphans()` ; - deduplica `samestock_products` conservando el `id` más bajo; - deduplica `samestock_combis` conservando el `id` **más alto** (la última elección real del usuario); - crea los índices `UNIQUE` y `KEY` si no existen (comprobando `INFORMATION_SCHEMA`).

El orden importa: los `UNIQUE` fallarían si quedaran duplicados.

`purgeOrphans()` elimina filas con ids a 0 o autorreferenciadas, filas de productos inexistentes y —lo más importante— filas cuya combinación origen o destino **no pertenece** al producto declarado. Esa última consulta es la que repara la corrupción dejada por las versiones anteriores.

10. Sistema de depuración

`SameStockLogger` , activado por `SAMESTOCK_DEBUG` .

- Ficheros diarios `logs/samestock-AAAA-MM-DD.log`.
- Cada línea lleva marca de tiempo y un identificador de petición de 8 caracteres, para poder seguir un guardado completo.
- `logs/.htaccess` deniega el acceso web y desactiva el motor PHP en la carpeta.
- Se registran: el payload crudo de `asociaciones`, la lista CSV de productos, los pares normalizados, los payloads rechazados por incoherencia, cada propagación de stock con los nodos alcanzados, `purgeOrphans`, `migrate` y los errores.
- El logger nunca lanza excepciones: cualquier fallo de escritura se ignora.

11. Notas de compatibilidad verificadas contra el core

Verificado leyendo las fuentes de 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1 en `C:\dev-tools\prestashop`.

Propagación de stock en la página V2

`StockAvailable::setQuantity()` (que es donde vive el `Hook::exec('actionUpdateQuantity')` de `classes/stock/StockAvailable.php:434`) **no se usa** en la página V2: ésta escribe con `StockAvailableRepository::update()`, que es un `ObjectModel::update()` y no pasa por ahí.

Si ése fuera el único punto de disparo, el módulo no propagaría nada en la V2. No lo es: la V2 lanza el hook por su cuenta, con los mismos parámetros.

Punto de disparo	Contexto
<code>classes/stock/StockAvailable.php:434</code> (<code>setQuantity</code>)	página clásica, pedidos, código legacy
<code>src/Adapter/Product/Stock/Update/ProductStockUpdater.php:298</code>	V2, producto sin combinaciones
<code>src/Adapter/Product/Combination/Update/CombinationStockUpdater.php:157</code>	V2, rejilla de combinaciones
<code>src/Core/Stock/StockManager.php:169</code>	movimientos de stock

Parámetros en los tres casos: `id_product`, `id_product_attribute`, `quantity`, `delta_quantity` (sólo V2) e `id_shop`. En `CombinationStockUpdater` el `id_shop` puede ser **0** cuando el grupo de tiendas comparte stock; por eso `applyQuantity()` filtra con `id_shop IN (...) OR id_shop = 0` y cae en `getSourceShops()` cuando llega 0.

Los dos updaters de la V2 sólo llaman a `updatePhysicalProductQuantity()` para la fila que el empleado ha editado, no para los nodos a los que propagamos; `applyQuantity()` recalcula por eso `physical_quantity = quantity + reserved_quantity` en los nodos que toca. La columna existe en 1.7.8 y en 9.1 (verificado en `db_structure.sql`), y de todas formas se comprueba antes con `columnExists()`.

API por versión

Elemento	1.7.8	8.1 / 8.2	9.1
<code>Product::getAttributesResume(\$id_lang)</code>	sí	sí	sí (<code>Product.php:2463</code>)
<code>actionProductUpdate</code> con <code>['id_product','product']</code>	sí	sí	sí (<code>Product.php:790</code>)
<code>actionUpdateQuantity</code> con <code>id_shop</code>	sí (<code>StockAvailable.php:217</code>)	sí	sí
<code>deleteProductAttribute</code> (borrar UNA combinación)	sí (<code>Product.php:2636</code>)	sí	sí (<code>Product.php:2396</code>)
<code>actionProductAttributeDelete</code> (borrar TODAS)	sí	sí	sí
<code>HelperForm</code> con <code>tabs</code>	sí	sí	sí
<code>Configuration::getGlobalValue</code> / <code>updateGlobalValue</code>	sí	sí	sí
<code>Module::trans()</code>	sí, protected	público	público
<code>SymfonyContainer::getInstance()</code>	sí	sí	sí
tabla <code>feature_flag</code>	no	sí	sí
<code>_legacy_controller: AdminProducts</code> en las rutas	sí	sí	sí

Notas:

- `Module::trans()` es **protected** en 1.7.8: sólo se invoca desde dentro de la clase.
- `Hook::registerHook()` crea la fila del hook si el nombre no existe todavía (verificado en 1.7.8), así que registrar `deleteProductAttribute` es seguro. Aun así, en `install()` sólo son bloqueantes `displayBackOfficeHeader`, `actionUpdateQuantity` y `actionProductUpdate`; el resto se registra en modo "mejor esfuerzo" para no tumbar la instalación.
- `isProductPage()` usa como señal principal `_legacy_controller` de la petición Symfony (vale para las tres versiones), con el nombre de ruta, `Tools::getValue('controller')` y el objeto controlador del contexto como respaldos.
- PHP mínimo de 1.7.8: 7.1.3, así que `Throwable` está disponible. Los `catch` defensivos usan `Throwable` porque en PHP 8 un método o clase inexistente lanza `Error`, que no es `Exception`.
- No se usa `$controller::class`, ni `namespace`, ni Composer, ni `installOverrides()`.
- Sin `LIMIT 1` añadido a mano. Todos los ids van con `(int)`; `pSQL()` sólo en cadenas.

Limitaciones conocidas

- Al añadir un producto asociado hay que **guardar** antes de poder elegir sus combinaciones: los desplegables se construyen al renderizar la página.

- La lista de productos se carga completa (idioma y tienda actuales) en `Media::addJsDef`. En catálogos muy grandes conviene pasar a búsqueda por AJAX.
- Con productos de muchísimas combinaciones el HTML crece: se generan `combinaciones_propias × (combinaciones_asociadas + 1)` elementos `<option>` por producto asociado. Con volúmenes normales (decenas) es irrelevante.
- En 1.7.8 existe `Admin/Sell/Catalog/Product/edit.html.twig`, pero ningún controlador lo usa y no renderiza los hooks V1: es código dormido. La página activa en 1.7.8 es `Admin/Product/ProductPage/product.html.twig`.

samestock — Documentación de desarrollo

Versión: 1.6.4 · **Autor:** Ecom Experts ecomyseo@gmail.com · **Licencia:** AFL-3.0

Este documento explica **qué estaba roto, por qué, y cómo se arregló**. Sirve como registro para no volver a introducir los mismos fallos.

Síntomas reportados

1. Al asociar más de 3 productos, deja de guardar bien.
2. Al asociar más de 2 combinaciones, deja de guardar bien.
3. Al elegir una combinación que ya estaba usada por otra, se mezclan.

No era un único fallo: eran **seis** causas distintas que se solapaban.

Causa 1 — El JS concatenaba el id en el `name` en vez de reemplazarlo

La causa principal. En `views/js/combis.js`:

```
var name = $(this).attr('name');
$(this).attr('name', name + '[' + id_product_attribute + ']');
```

La plantilla ya renderizaba el campo con el destino guardado dentro del nombre:

```
asociaciones[ORIGEN][PRODUCTO][DESTINO_ANTIGUO]
```

Al elegir una combinación quedaba:

```
asociaciones[ORIGEN][PRODUCTO][DESTINO_ANTIGUO][DESTINO_NUEVO]
```

Y eligiendo dos veces, tres niveles. El PHP sólo leía tres niveles (`foreach($asociaciones as $pa => $values) → $id_pro => $combinaciones → $id_pro_attribute`), así que tomaba `DESTINO_ANTIGUO` como destino y **descartaba silenciosamente el elegido**. De ahí el "elijo una combinación y no la guarda".

Arreglo: el destino ya no viaja en el nombre. Va en el **valor** de un campo oculto de nombre **fijo** `asociaciones[ORIGEN][PRODUCTO]` . El campo de texto visible no tiene `name` . No se manipula ningún `name` en ningún momento, así que este fallo es imposible por construcción.

Causa 2 — El autocompletar ofrecía las combinaciones de todos los productos

`hookdisplayBackOfficeHeader` construía `combisposibles` como una lista **plana** con las combinaciones de todos los productos asociados, y `combis.js` la pasaba entera como `source` de **todos** los campos.

Cada campo pertenece a un producto concreto (su nombre lleva `[id_pro]` fijo), así que se podía elegir una combinación del producto B en el campo del producto A y se guardaba la fila (`id_pro = A, id_pro_attribute = <combinación de B>`). Corrupción directa, y tanto más probable cuanto más productos asociados: **el fallo a partir de 3 productos**.

Además, la entrada de la lista tenía un bug: `'id_product' => $id_pro`, donde `$id_pro` era la **fila entera** del `ExecuteS`, no el id.

Arreglo: cada entrada lleva su `id_pro`, el JS agrupa por producto y cada campo sólo recibe las combinaciones de su producto. Y como defensa en el servidor, `normalizeCombinationPayload()` verifica contra `product_attribute` que la combinación destino pertenece realmente al producto asociado; si no, la descarta y lo registra.

También: `if(combisposibles)` era siempre verdadero porque `[]` es *truthy* en JS, así que el `else` con la lista de productos nunca se ejecutaba. Ahora se comprueba `.length`.

Causa 3 — El `DELETE` de guardado usaba la columna equivocada

En `hookActionProductUpdate`:

```
$sql = "delete from ..._samestock_combis
      where id_pro_attribute = ".$id_pro_attribute."
      and id_product = ".$id_product;
```

Borraba por `id_pro_attribute` (el **destino**) en lugar de por `id_product_attribute` (el **origen**). Si la combinación 5 del producto 10 apuntaba a 99 y el usuario la cambiaba a 101, el `DELETE` buscaba `id_pro_attribute = 101`, que no coincidía con la fila antigua (`id_product_attribute = 5, id_pro_attribute = 99`). **La fila antigua sobrevivía** y quedaban dos: 5→99 y 5→101.

Luego `hookActionUpdateQuantity` recorría las dos y escribía cantidades distintas en secuencia. Eso es exactamente el "se mezclan combinaciones" y el "no limpia".

Arreglo: `saveCombinationAssociations()` borra por la clave de **origen** (`id_product, id_product_attribute, id_pro`) en los dos sentidos, libera además la combinación destino de cualquier otro origen, y sólo entonces inserta. Nunca hay `UPDATE`. Y hay dos índices `UNIQUE` que impiden el estado incorrecto a nivel de base de datos.

Causa 4 — Comprobación anti-duplicado tautológica

```
$sqlch = "select id_pro_attribute from ..._samestock_combis
      where id_product_attribute = ".$id_pro_attribute."
      and id_product_attribute = ".$id_product_attribute;
```

La **misma columna** comparada con dos valores distintos: la condición es siempre falsa salvo que los dos ids coincidan. `$id_att` era siempre `null`, así que la fila inversa se insertaba **incondicionalmente en cada guardado**, acumulando duplicados indefinidamente. Segunda fuente del "se mezclan".

Arreglo: eliminada. La secuencia borrar → insertar es idempotente por sí misma, y los índices `UNIQUE` son el respaldo.

Causa 5 — Colisión de claves en las filas nuevas de productos

En `views/js/samestock.js`:

```
indexid = $('[id^="samestock_"]').length + 1;
... name="idproductosasociados['+indexid+']"
```

Las filas existentes usaban el **id del producto** como clave (`idproductosasociados[{$asociado.id_pro}]`) y las nuevas un **contador**. Con 3 o 4 productos asociados el contador acababa coincidiendo con el id de un producto ya asociado, las dos filas compartían nombre de array y **una machacaba a la otra**. Otra pieza del "a partir de 3 productos".

En el mismo fichero había tres bugs más:

- `items.push(...)` se ejecutaba sobre `pros` en **cada** click de "Añadir", así que a la cuarta fila el desplegable mostraba cada producto repetido cuatro veces.
- Los `focusout` y los `click` de borrar se volvían a enlazar **dentro** del click, duplicando handlers en cada pulsación.
- El borrar hacía `$("#samestock_" + id_product).remove()`; en las filas nuevas `id_product` valía `0`, así que apuntaba a `#samestock_0`, que no existe: **la fila nunca se borraba**.

Arreglo: la lista completa se mantiene en **un solo** campo oculto CSV (`samestock_products_list`) que el JS recalcula en cada cambio; se eliminan las claves por fila y con ellas las colisiones. El `source` se construye una vez. Los eventos se delegan sobre `document` una única vez. El borrado usa `closest('.samestock-row')`.

Causa 6 — Recorrido de propagación escrito a mano y de sólo 2 niveles

`hookActionUpdateQuantity` recorría las combinaciones directas y luego hacía una segunda pasada sobre el array `$ups`. Con cadenas de más de 3 productos, o más de 2 combinaciones encadenadas, el grafo quedaba **sin cubrir**. Además reutilizaba `$stockpr` en la segunda pasada y sobrescribía `$id_product` / `$id_product_attribute` dentro del bucle, contaminando el bloque de propagación de productos que venía después.

Arreglo: `getLinkedCombinations()` y `getLinkedProducts()` calculan el **cierre transitivo** completo con recorrido en anchura sobre las dos direcciones de la relación, conjunto de visitados y tope `MAX_NODES = 300`.

Fallos adicionales corregidos por el camino

Problema	Detalle	Arreglo
Fatal en PHP 8	<code>foreach(\$asociaciones ...)</code> con <code>\$asociaciones === false</code> (producto guardado sin asociaciones)	se comprueba <code>is_array()</code>
Borrado accidental masivo	si <code>idproductosasociados</code> no era array, se borran todas las asociaciones del producto. Un guardado por webservice o desde cualquier otra pantalla las destruía	sólo se actúa si el campo de la sección llega en el POST
SQL injection / error de sintaxis	<code>\$id_product</code> interpolado en crudo desde <code>\$request->get('id')</code> ; si era <code>null</code> , <code>id_product !=</code> daba error de sintaxis	todo con <code>(int)</code> y con guarda de <code>id > 0</code>
<code>pSQL()</code> sobre enteros	<code>pSQL(\$id_product)</code> en lugar de <code>(int)</code>	corregido en todas las consultas
Fatal por <code>global \$kernel</code>	<code>\$kernel->getContainer()</code> sin comprobar que <code>\$kernel</code> existe (falla en AJAX legacy y CLI)	<code>getSymfonyRequest()</code> usa <code>SymfonyContainer::getInstance()</code> con <code>\$GLOBALS['kernel']</code> como respaldo, todo en <code>try/catch (Throwable)</code>
Total del producto desfasado	nunca se recalculaba la fila <code>id_product_attribute = 0</code>	<code>recomputeParentQuantity()</code> , opcional
Producto con combinaciones pisado	se le escribía un stock global plano al propagar una combinación	se salta: su total es la suma de sus combinaciones
Multitienda: duplicados	el autocompletar leía <code>product_lang</code> sin filtrar por <code>id_shop</code>	filtro por la tienda del contexto
Multitienda: stock	los <code>UPDATE</code> de <code>stock_available</code> no filtraban por tienda	se usa el <code>id_shop</code> del hook, o las tiendas de la fila origen
<code>Undefined array key</code> en Smarty	la plantilla accedía a <code>\$combis[\$id_pro][\$id_attribute]</code> sin garantizar la clave	el PHP construye la matriz completa con todos los valores por defecto
Filas huérfanas al borrar	no había hooks de borrado	<code>actionProductDelete</code> , <code>actionProductAttributeDelete</code> y <code>deleteProductAttribute</code>
Traducciones mezcladas	<code>listacombis.tpl</code> usaba el sistema legacy <code>mod='samestock'</code>	fichero eliminado (era código muerto: usaba variables Smarty que ningún PHP asignaba)
Carpeta prohibida	<code>upgrade/upgrade-1.1.0.php</code> era un stub vacío	eliminada; las migraciones van en <code>addnewfeatures()</code>

Segunda revisión: dos blockers más de la página V2

Tras la primera tanda de correcciones se revisó de nuevo contra las fuentes de 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1. Aparecieron dos problemas más y un riesgo.

Blocker A — ¿llegaba siquiera el hook de stock en la V2?

`Hook::exec('actionUpdateQuantity')` vive en `StockAvailable::setQuantity()` (`classes/stock/StockAvailable.php:434`). La página V2 **no pasa por ahí**: escribe con `StockAvailableRepository::update()`, que es un `ObjectModel::update()`. Si ése fuera el único punto de disparo, el módulo no propagaría absolutamente nada en la V2.

Comprobado: **no lo es**. La V2 lanza el hook por su cuenta, en `ProductStockUpdater.php:298` (producto sin combinaciones) y en `CombinationStockUpdater.php:157` (rejilla de combinaciones), con los mismos parámetros más `delta_quantity`. La propagación funciona sin cambios.

Un detalle sí requirió cambio: en `CombinationStockUpdater` el `id_shop` que se pasa es `$stockAvailable->id_shop`, que vale **0** cuando el grupo de tiendas comparte stock. El código ya cae en `getSourceShops()` en ese caso y filtra con `id_shop IN (...) OR id_shop = 0`. Y los dos updaters recalculan `physical_quantity` **sólo** para la fila que el empleado editó, no para los nodos propagados: `applyQuantity()` lo recalcula ahora también en los nodos que toca, o los informes de stock quedan descuadrados.

Blocker B — jQuery UI no está garantizado en la V2

El módulo dependía de `$.autocomplete` (jQuery UI). La página V2 no lo carga de forma garantizada: `js/jquery/ui/jquery-ui.js` existe en el core pero no se incluye en el layout de producto. Sin él, el campo quedaba inerte, no se podía elegir nada y la pantalla era inservible — justo la funcionalidad que había que arreglar.

Arreglo, eliminando la dependencia en lugar de intentar cargarla:

- **Combinaciones** → `<select>` **nativo**. Cero JavaScript. Además refuerza la corrección del fallo original por dos vías: el destino va en el `value` (no en el `name`) y las opciones son sólo las combinaciones de ese producto asociado, así que elegir una combinación de otro producto es **imposible desde el navegador**. Y al cambiar un `<select>` el navegador emite `change`, que es el evento que escucha el gestor de envío parcial de la V2 para habilitar el botón de guardar. `views/js/combis.js` se ha eliminado.
- **Lista de productos** → **JavaScript nativo** con lista de sugerencias propia (`samestock.js` + `samestock.css`). Sin jQuery ni jQuery UI.

Verificado además que sí llegan a la V2 las dos cosas que el módulo necesita del hook de cabecera:

`head_tag.html.twig` renderiza `displayBackOfficeHeader` **antes** de los bucles de `this.cssFiles`, `this.jsDef` y `this.jsFiles`, así que `addJS()`, `addCSS()` y `Media::addJsDef()` funcionan igual que en la página clásica. Aun así, las llamadas al controlador van protegidas con `method_exists()`.

Riesgo cerrado — `actionProductUpdate` de otro producto

`actionProductUpdate` no se lanza sólo desde `Product::update()`: también desde `updatePosition()` (reordenar el catálogo), `setWsCategories()` y `updateWs()` (webservice). Si uno de esos disparos coincidiera en la misma petición que nuestro POST, `saveProductAssociations()` habría aplicado el formulario al producto equivocado.

Dos cambios:

1. El id de producto se toma de `$params['id_product']` que aporta PrestaShop (autoritativo), y sólo como respaldo de `Tools::getValue('id_product')`. Antes era al revés, así que un `id_product` residual en la URL podía

ganar.

2. La lista viaja indexada por producto: `samestock_products_list[ID_PRODUCTO]`. El nombre sigue siendo fijo para una página dada (compatible con el envío parcial) pero ahora está acotado. `extractProductListForProduct()` devuelve `null` si la clave no corresponde, y entonces no se toca nada.

Para las combinaciones la protección ya era implícita y demostrable: `normalizeCombinationPayload()` descarta todo `src_pa` que no pertenezca al producto.

Verificado también para PS 1.7.8

- No existe la tabla `feature_flag`, así que `isProductPageV2()` devuelve `false` y se usa la página clásica. Correcto.
- `Module::trans()` es `protected` en 1.7.8 (es público en 8.x/9.x). Sólo se invoca desde dentro de la clase, así que sirve igual.
- `HelperForm` soporta `tabs` en `admin/themes/default/template/helpers/form/form.tpl`.
- `Hook::registerHook()` crea la fila del hook si el nombre no existe, así que registrar `deleteProductAttribute` es seguro. Aun así `install()` sólo hace bloqueantes los tres hooks imprescindibles y registra el resto en modo "mejor esfuerzo", para no tumbar la instalación por un hook accesorio.
- `Admin/Sell/Catalog/Product/edit.html.twig` existe en 1.7.8 pero ningún controlador lo usa y no renderiza los hooks V1: es código dormido, no afecta.
- PHP mínimo 7.1.3 → `Throwable` disponible.

Detección de la página de producto, endurecida

`isProductPage()` usa ahora como señal principal `_legacy_controller` de la petición Symfony, que las rutas de producto declaran como `AdminProducts` en las tres versiones (verificado en `products/product.yml`, ruta `admin_products_edit`, y en `combination.yml`). Antes dependía de `Tools::getValue('controller')`, que en las rutas Symfony de la V2 no viene. Se conservan como respaldo el nombre de ruta, el parámetro `controller` y el objeto controlador del contexto.

Tercera revisión: "no sale nada en la ficha de producto"

Reportado con captura: en la ficha de producto sólo aparecía **Productos relacionados** (el campo nativo de la V2), y ninguna opción del módulo.

Causa — el hook no estaba registrado y en V2 eso significa no existir

En la página V2 el módulo sólo aparece si está registrado en `displayAdminProductsExtra`, porque `ProductTypeListener` **elimina** el campo `extra_modules` del formulario cuando ningún módulo implementa ese hook:

```
$registeredExtraModules = $this->hookInformationProvider->getRegisteredModulesByHookName(
    ExtraModulesType::HOOK_NAME
);
```

Al actualizar desde una versión antigua del módulo, ese hook no estaba registrado (la versión anterior no lo usaba), así que no había ni pestaña donde mirar. El módulo lo registra en `addnewfeatures()`, pero eso sólo corre **al abrir la**

configuración del módulo: hasta entonces, nada.

Arreglo: `autoRegisterHooks()`, llamado desde el constructor cuando el módulo se instancia en el back-office.

Registra los hooks que faltan y guarda la versión en `SAMESTOCK_HOOKS_VERSION`; en cuanto coincide, la comprobación es una lectura de `Configuration` ya en caché. Todo en `try/catch (Throwable)`.

Además la lista de hooks se ha centralizado en `Samestock::getHookList()`, que usan `install()`, `addnewfeatures()` y el auto-registro, para que no puedan desincronizarse. Y se ha quitado `actionAdminControllerSetMedia`, que estaba registrado desde la versión original **sin método que lo implementase**.

Y el sitio donde aparecía tampoco era el bueno

La pestaña *Módulos* funciona, pero no es donde el usuario espera encontrarlo. En la segunda revisión se había descartado crear una pestaña propia por fragilidad; al investigar el mecanismo real de las pestañas de la V2 resultó que hay una vía determinista, y se ha implementado.

Mecanismo, verificado en 8.1.7, 8.2.7 y 9.1.1:

1. `EditProductFormType::getParent()` devuelve `NavigationTabType`.
2. El bloque `navigation_tab_widget` de `Admin/TwigTemplateForm/prestashop_ui_kit_base.html.twig` (línea ~1408) recorre **todos los hijos de la raíz** del formulario y convierte cada uno en pestaña: `nav <li id="{{ child.vars.id }}-tab-nav">`, panel `<div id="{{ child.vars.id }}-tab">`, título `{{ child.vars.label }}`.
3. El prefijo del formulario raíz es `product` — de ahí que el propio core use `#product_stock-tab` en su `product-map.ts`.

Así que `hookActionProductFormBuilderModifier` añade un hijo llamado `samestock (FormType, mapped => false, required => false)` y el core genera `#product_samestock-tab` de forma determinista. **Ese id es nuestro**, no depende de la estructura interna del core, y por eso el traslado con JS sí es fiable aquí. El contenido lo sigue pintando `displayAdminProductsExtra` y `samestock.js` lo mueve a la pestaña.

Lo que antes descarté sigue descartado, y por los mismos motivos: colarse en `#product_stock-tab` no vale porque `ProductTypeListener` hace `$form->remove('stock')` en varios casos y esa pestaña no existe siempre. La diferencia es que ahora la pestaña la creamos nosotros.

Degradación en cascada, sin punto único de fallo:

Qué falla	Resultado
nada	pestaña Stock compartido con todo dentro
el JS no se ejecuta	contenido en la pestaña Módulos + una pestaña vacía
<code>actionProductFormBuilderModifier</code> sin registrar	contenido en la pestaña Módulos
no hay contenido que mover	el JS retira la pestaña para no dejarla vacía

`mapped => false` es importante: el módulo no guarda a través del formulario de Symfony, lee el POST crudo en `actionProductUpdate` (el patrón de "inyección directa" que describen los skills). Los campos van fuera del espacio de nombres `product[...]`, así que Symfony los ignora y `Tools::getValue()` los ve.

Verificado que el hook y el id son los correctos: `EditProductFormType::getBlockPrefix()` devuelve `'product'` en 8.2.7 y en 9.1.1. De ahí que el hook sea `actionProductFormBuilderModifier` (y no

`actionEditProductFormBuilderModifier`) y que la pestaña sea `#product_samestock-tab`. `CreateProductFormType` usa el prefijo `create_product`, así que la pantalla de creación no dispara nuestro hook.

Cuarta revisión: seguía sin salir nada

Reportado de nuevo con captura, y con un dato muy útil en ella: **otro módulo (Creative Elements) sí conseguía inyectar** su campo `DisplayFooterProduct` en el formulario. Es decir, el mecanismo funciona en esa tienda; el problema era del módulo.

Causa raíz — la detección de V1/V2 se deducía de la configuración

`isProductPageV2()` decidía así:

1. `version_compare(_PS_VERSION_, '9.0.0', '>=')`, o
2. leer `state` del flag `product_page_v2` en la tabla `feature_flag`.

Y de esa decisión dependía **todo**: si devolvía `false`, `displayAdminProductsExtra` devolvía cadena vacía y los hooks V1 intentaban pintar. Pero en la página V2 los hooks V1 **no se renderizan nunca**. Resultado: si la deducción falla, no se pinta absolutamente nada, en ningún sitio. Un único punto de fallo, y basado en una suposición.

Y hay muchas formas de que falle en una tienda real: una 8.1/8.2 actualizada desde 8.0 puede no tener la fila del flag, o tenerla a 0 mientras la página que se sirve es la V2.

Arreglo: dejar de deducir y detectar en ejecución. El hook `actionProductFormBuilderModifier` **sólo puede ejecutarse en V2**, y eso está verificado en el código fuente, no supuesto:

- El `ProductController` de V1 construye su formulario con el `createFormBuilder()` de Symfony y tiene **cero** referencias a `IdentifiableObject` (comprobado con `grep -c` en 8.2.7).
- El único que despacha `action<Prefijo>FormBuilderModifier` es `Core\Form\IdentifiableObject\Builder\FormBuilder::buildForm()`.
- El `ProductController` de V2 sí usa `prestashop.core.form.identifiable_object.builder.edit_product_form_builder`.

Así que en cuanto ese hook se ejecuta, ponemos `self::$v2_runtime = true`. Y ocurre en tiempo de **construcción** del formulario, antes de cualquier renderizado, así que `displayAdminProductsExtra` ya lo encuentra puesto. `isProductPageV2()` devuelve ahora `$v2_runtime || versión ≥ 9 || flag`: la señal de ejecución gana, y la deducción queda sólo como respaldo.

Un camino que se descartó (y por qué)

La alternativa "que `displayAdminProductsExtra` pinte sólo si los hooks V1 no han pintado" **no sirve**: en el `product.html.twig` de V1, `displayAdminProductsExtra` se renderiza en la **línea 33**, antes de los paneles (líneas 68 y 91). Cuando se ejecuta, todavía no se sabe si los paneles van a pintar. Comprobado en 8.2.7.

Panel de diagnóstico

Tres iteraciones con el mismo síntoma y sin poder reproducir el entorno. Se añade un panel en la configuración del módulo que muestra de un vistazo: versión del módulo, de PrestaShop y de PHP; **página de producto detectada (V1/V2)**; existencia de la tabla `feature_flag` y valor del flag; multitienda; versión con hooks ya registrados; si

`logs/` es escribible; número de filas en las tablas; presencia de los índices `UNIQUE` ; y **la lista de los 10 hooks con una marca de registrado/no registrado**.

Una captura de ese panel identifica la causa sin más idas y venidas. Además los tres hooks de pintado registran ahora en el log su decisión y por qué (`es_v2` , `v2_en_ejecucion` , `ps_version` , `pinta`).

Compatibilidad PS 8 / PS 9: el blocker que había que resolver

Verificado contra las fuentes de 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1:

`displayAdminProductsMainStepLeftColumnBottom` y `displayAdminProductsQuantitiesStepBottom` son hooks de la página de producto **clásica (V1)**.

- En 1.7.x y 8.0 se renderizan.
- En 8.1 y 8.2 sólo si el feature flag `product_page_v2` está **apagado** (viene activado de fábrica: `feature_flag.xml` , `state="1"`).
- En 9.1 **la página V1 ya no existe**: el directorio `src/PrestaShopBundle/Resources/views/Admin/Product/` fue eliminado. Los hooks siguen en `hook.xml` (el `registerHook` funciona sin error) pero **ninguna plantilla del core los invoca**. El único hook `displayAdminProducts*` que PS 9 renderiza es `displayAdminProductsExtra` .

Es decir: **el módulo no mostraba nada en PS 9**, y tampoco en PS 8.1/8.2 con la configuración de fábrica. Arreglar sólo el guardado habría dejado la corrección invisible.

Arreglo: se añade `hookDisplayAdminProductsExtra` , y `isProductPageV2()` decide qué vía usar (leyendo `feature_flag.state` , o `true` directo si `_PS_VERSION_ >= 9.0.0`). Los tres hooks de display son mutuamente excluyentes: nunca se renderizan los campos dos veces. En V2 el contenido aparece en la pestaña **Módulos**, que sí está dentro del `<form>` del producto (`ExtraModulesType` es un campo de `EditProductFormType`).

La trampa del envío parcial en V2

`product-partial-updater.ts` toma una instantánea del formulario al cargar y al guardar envía **sólo los campos cuyo valor cambió**, en un formulario "sombra" con método `PATCH` . Consecuencias:

- Un dato codificado en el `name` es **invisible** para ese diff.
- Una lista repartida en varios campos llegaría **incompleta**, y con semántica de "borrar y reconstruir" se habrían borrado asociaciones que el usuario no tocó.
- Los campos presentes en la instantánea y ausentes al guardar se fuerzan a `''` .

Por eso el rediseño del transporte: nombres fijos, valores variables, y la lista de productos en un único campo CSV. Tras cada cambio el JS hace `trigger('change')` para que el updater detecte la modificación (escucha `keyup change dp.change` en el formulario) y habilite el botón de guardar.

Verificación realizada

- `php -l` sobre todos los `.php` y `node --check` sobre los dos `.js` : sin errores.
- **Banco de pruebas** con base de datos falsa en memoria (`scratchpad/test_samestock.php`), **43 aserciones, todas correctas**, incluyendo:

- la guardia por producto de `extractProductListForProduct()` : lista de otro producto → `null` , lista vacía del propio → desasociar todo, clave como cadena (como llega de `$_POST`), campo ausente → `null` ;
- payload con la forma actual (escalar, el `value` del `<select>`), con `0` y vacío;
- payload corrupto de la versión antigua con 2 y 3 índices concatenados → toma la elección real;
- escenario de envío parcial de V2 (`[nuevo => dato, viejo => '']`) en los dos órdenes posibles;
- combinación de otro producto → rechazada;
- 4 productos × 3 combinaciones → 18 filas simétricas;
- cambiar una combinación a un destino **ya usado por otra** → el destino se libera, la antigua desaparece, el espejo correcto se crea, y las asociaciones con los otros productos no se tocan;
- invariante "ningún origen con dos destinos del mismo producto";
- cierre transitivo sobre una cadena de 4 productos, desde los dos extremos;
- desasociar un producto limpia sus combinaciones y no toca las demás.
- **Script determinista** de extracción de cadenas (`scratchpad/check_strings.php`): las 40 cadenas del código están en `es-ES.xlf` con el dominio correcto, el XML es válido, y no queda ninguna cadena con la sintaxis legacy `mod='samestock'` .
- Hooks, firmas y parámetros verificados leyendo el código fuente de PS 1.7.8.11, 8.1.7, 8.2.7 y 9.1.1 en `C:\dev-tools\prestashop` .

Lo que **no** se ha podido verificar sin una tienda real: el comportamiento en el navegador (autocompletar, pestaña Módulos en V2) y la propagación con stock real. Para eso está el modo depuración.

Si algo falla en producción

1. Configuración del módulo → pestaña **Depuración** → activar **Modo depuración** → Guardar.
2. Reproducir el caso: asociar los productos, elegir las combinaciones, guardar.
3. Enviar `modules/samestock/logs/samestock-AAAA-MM-DD.log` .

El log contiene el payload crudo recibido, los pares normalizados, lo que se descartó y por qué, y los nodos alcanzados en cada propagación. Con eso se distingue en un minuto si el problema está en el navegador, en el transporte o en la base de datos.